

Introducción al Repository Pattern



Objetivos

- Entender qué es el Repository Pattern y su propósito
- Conocer los problemas que resuelve
- Ver la diferencia entre acceso directo a DB y usar repositorios
- Comprender cuándo aplicar este patrón

¿Qué es el Repository Pattern?

El **Repository Pattern** es un patrón de diseño que **abstrae el acceso a datos** detrás de una interfaz similar a una colección. El resto de la aplicación trabaja con objetos de dominio sin conocer los detalles de cómo se almacenan.

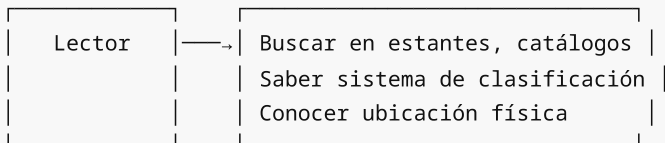
Definición de Martin Fowler

"Un repositorio media entre el dominio y las capas de mapeo de datos usando una interfaz similar a una colección para acceder a objetos del dominio."

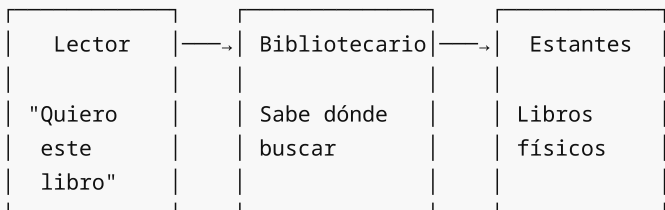
Analogía: El Bibliotecario

Imagina una biblioteca:

Sin repositorio (acceso directo):



Con repositorio (bibliotecario):



El lector (service) solo pide el libro. El bibliotecario (repository) sabe cómo encontrarlo.

Arquitectura en Capas



Problema: Sin Repository Pattern

Código Típico de Week-06

En la semana anterior, nuestros servicios accedían directamente a SQLAlchemy:

```
# services/author_service.py - Week 06
from sqlalchemy import select
from sqlalchemy.orm import Session

from models import Author

class AuthorService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        # ❌ Service conoce detalles de SQLAlchemy
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        # ❌ Service construye queries
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        # ❌ Lógica de paginación mezclada
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, data: AuthorCreate) -> Author:
        # Validación de negocio ✅
        if self.get_by_email(data.email):
            raise DuplicateError("Email already exists")

        # ❌ Operaciones de persistencia en service
        author = Author(**data.model_dump())
        self.db.add(author)
        self.db.commit()
        self.db.refresh(author)
        return author
```

Problemas de este Enfoque

Problema	Descripción
Acoplamiento	Service está acoplado a SQLAlchemy
Testing difícil	Necesitas BD real para testear lógica de negocio
Duplicación	Mismas queries en múltiples services
Cambio costoso	Cambiar de ORM requiere modificar todos los services
Responsabilidades mezcladas	Lógica de negocio + acceso a datos

✅ Solución: Repository Pattern

Separación de Responsabilidades

```

# repositories/author_repository.py
from sqlalchemy import select
from sqlalchemy.orm import Session

from models import Author

class AuthorRepository:
    """Repositorio para operaciones de datos de Author"""

    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def add(self, author: Author) -> Author:
        self.db.add(author)
        self.db.flush() # Obtiene ID sin commit
        return author

    def delete(self, author: Author) -> None:
        self.db.delete(author)

```

```

# services/author_service.py - CON Repository
from models import Author
from schemas import AuthorCreate
from repositories import AuthorRepository
from exceptions import DuplicateError

class AuthorService:
    """Service para lógica de negocio de Author"""

    def __init__(self, author_repo: AuthorRepository):
        # ✅ Recibe repository, no Session
        self.repo = author_repo

    def get_by_id(self, author_id: int) -> Author | None:
        # ✅ Delega al repository
        return self.repo.get_by_id(author_id)

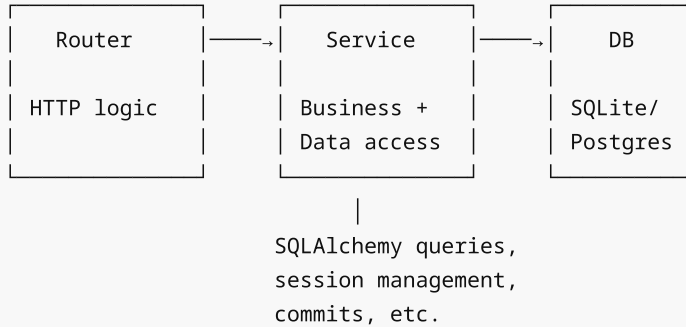
    def create(self, data: AuthorCreate) -> Author:
        # ✅ Solo lógica de negocio
        if self.repo.get_by_email(data.email):
            raise DuplicateError("Email already exists")

```

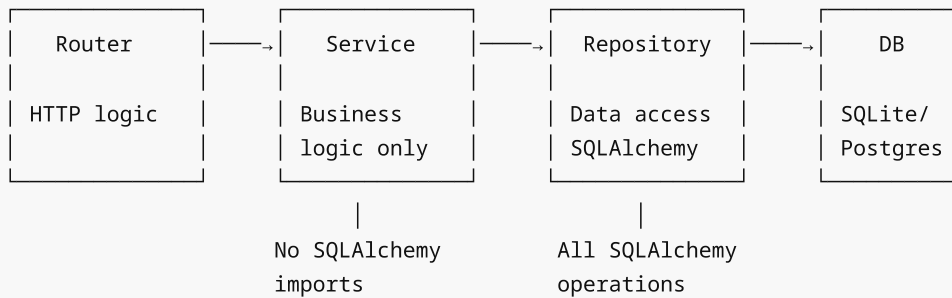
```
# ✅ Crea objeto y delega persistencia
author = Author(**data.model_dump())
return self.repo.add(author)
```

Comparación Visual

Antes (Week-06): Service → Database



Después (Week-07): Service → Repository → Database



Beneficios del Repository Pattern

1. Separación de Responsabilidades

```
# Service: SOLO lógica de negocio
class TaskService:
    def complete_task(self, task_id: int) -> Task:
        task = self.repo.get_by_id(task_id)
        if not task:
            raise NotFoundError("Task not found")

        # Regla de negocio
        if task.is_completed:
            raise BusinessError("Task already completed")

        task.is_completed = True
        task.completed_at = datetime.utcnow()
        return self.repo.update(task)

# Repository: SOLO acceso a datos
class TaskRepository:
    def get_by_id(self, task_id: int) -> Task | None:
```

```

        return self.db.get(Task, task_id)

def update(self, task: Task) -> Task:
    self.db.flush()
    return task

```

2. Testing Simplificado

```

# Test SIN base de datos real
class FakeTaskRepository:
    def __init__(self):
        self.tasks = {1: Task(id=1, title="Test", is_completed=False)}

    def get_by_id(self, task_id: int) -> Task | None:
        return self.tasks.get(task_id)

    def update(self, task: Task) -> Task:
        self.tasks[task.id] = task
        return task

def test_complete_task():
    fake_repo = FakeTaskRepository()
    service = TaskService(task_repo=fake_repo)

    task = service.complete_task(1)

    assert task.is_completed is True

```

3. Reutilización de Queries

```

# Repository con métodos específicos reutilizables
class TaskRepository:
    def get_pending_by_user(self, user_id: int) -> list[Task]:
        stmt = (
            select(Task)
            .where(Task.user_id == user_id)
            .where(Task.is_completed == False)
            .order_by(Task.due_date)
        )
        return self.db.execute(stmt).scalars().all()

    def get_overdue(self) -> list[Task]:
        stmt = (
            select(Task)
            .where(Task.due_date < date.today())
            .where(Task.is_completed == False)
        )
        return self.db.execute(stmt).scalars().all()

```

4. Facilita Cambios de Tecnología

```
# Si cambias de SQLAlchemy a otro ORM, solo cambias repositories
# Los services NO se modifican

# Repository con MongoDB (hipotético)
class TaskRepositoryMongo:
    def __init__(self, collection):
        self.collection = collection

    def get_by_id(self, task_id: str) -> Task | None:
        doc = self.collection.find_one({"_id": task_id})
        return Task(**doc) if doc else None
```

✗ Cuándo NO Usar Repository Pattern

El patrón agrega complejidad. No siempre es necesario:

Situación	¿Usar Repository?
Aplicación simple (CRUD básico)	✗ Probablemente no
Prototipo o MVP rápido	✗ No
Lógica de negocio compleja	✓ Sí
Múltiples fuentes de datos	✓ Sí
Testing unitario extensivo	✓ Sí
Equipo grande	✓ Sí
Posible cambio de ORM/DB	✓ Sí

🔗 Responsabilidades por Capa

Capa	Responsabilidad	Conoce
Router	HTTP, validación de request, responses	Schemas, HTTPException
Service	Lógica de negocio, validaciones, orquestación	Models, Repositories
Repository	CRUD, queries, persistencia	SQLAlchemy, Session

✓ Checklist de Comprensión

Antes de continuar, asegúrate de entender:

- ☐ El Repository Pattern abstrae el acceso a datos
- ☐ Services NO deben tener código SQLAlchemy
- ☐ Repositories manejan SOLO operaciones de datos
- ☐ El patrón facilita el testing con mocks/fakes
- ☐ No siempre es necesario (evaluar complejidad)

 **Siguiente**

En el próximo archivo aprenderemos a crear un **Repositorio Genérico** que evite duplicación de código entre repositorios específicos.

→ [02-repositorio-generico.md](#)

Repositorio Genérico (BaseRepository)

Objetivos

- Implementar un repositorio genérico reutilizable
- Usar Python Generics para tipado fuerte
- Evitar duplicación de código CRUD
- Crear la base para repositorios específicos

El Problema: Código Duplicado

Sin un repositorio genérico, cada entidad tiene código repetido:

```
# ❌ Duplicación en cada repositorio
class AuthorRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, id: int) -> Author | None:
        return self.db.get(Author, id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Author]:
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, entity: Author) -> Author:
        self.db.add(entity)
        self.db.flush()
        return entity

    def delete(self, id: int) -> bool:
        entity = self.get_by_id(id)
        if entity:
            self.db.delete(entity)
            return True
        return False

class PostRepository:
    def __init__(self, db: Session):
        self.db = db

    # ❌ Mismo código, diferente modelo
    def get_by_id(self, id: int) -> Post | None:
        return self.db.get(Post, id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Post]:
        stmt = select(Post).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()
```

```
# ... mismo patrón repetido
```

✓ Solución: BaseRepository con Generics

Python Generics

Los **Generics** permiten crear clases que trabajan con cualquier tipo:

```
from typing import TypeVar, Generic

# TypeVar define un tipo variable
T = TypeVar("T")

# Generic[T] indica que la clase usa ese tipo
class Container(Generic[T]):
    def __init__(self, item: T):
        self.item = item

    def get(self) -> T:
        return self.item

# Uso con tipos específicos
int_container = Container[int](42)
str_container = Container[str]("hello")
```

Implementación de BaseRepository

```
# repositories/base.py
from typing import TypeVar, Generic
from sqlalchemy import select, func
from sqlalchemy.orm import Session

from database import Base

# T debe ser un modelo SQLAlchemy (subclase de Base)
T = TypeVar("T", bound=Base)

class BaseRepository(Generic[T]):
    """
    Repositorio genérico con operaciones CRUD básicas.

    Uso:
        class AuthorRepository(BaseRepository[Author]):
            pass
    """

    def __init__(self, db: Session, model: type[T]):
        """
        Args:
            db: Sesión de SQLAlchemy
            model: Clase del modelo (Author, Post, etc.)
        """
```



```
self.db = db
self.model = model

def get_by_id(self, id: int) -> T | None:
    """Obtiene entidad por ID"""
    return self.db.get(self.model, id)

def get_all(self, skip: int = 0, limit: int = 100) -> list[T]:
    """Lista entidades con paginación"""
    stmt = select(self.model).offset(skip).limit(limit)
    return list(self.db.execute(stmt).scalars().all())

def count(self) -> int:
    """Cuenta total de entidades"""
    stmt = select(func.count()).select_from(self.model)
    return self.db.execute(stmt).scalar() or 0

def add(self, entity: T) -> T:
    """Agrega entidad a la sesión"""
    self.db.add(entity)
    self.db.flush() # Obtiene ID sin commit
    self.db.refresh(entity)
    return entity

def add_many(self, entities: list[T]) -> list[T]:
    """Agrega múltiples entidades"""
    self.db.add_all(entities)
    self.db.flush()
    for entity in entities:
        self.db.refresh(entity)
    return entities

def update(self, entity: T) -> T:
    """Actualiza entidad (ya debe estar en sesión)"""
    self.db.flush()
    self.db.refresh(entity)
    return entity

def delete(self, entity: T) -> None:
    """Elimina entidad"""
    self.db.delete(entity)
    self.db.flush()

def delete_by_id(self, id: int) -> bool:
    """Elimina por ID, retorna True si existía"""
    entity = self.get_by_id(id)
    if entity:
        self.delete(entity)
        return True
    return False

def exists(self, id: int) -> bool:
    """Verifica si existe entidad con ID"""
    return self.get_by_id(id) is not None
```

Creando Repositorios Específicos

Con `BaseRepository` , crear repositorios específicos es simple:

```
# repositories/author_repository.py
from models import Author
from repositories.base import BaseRepository

class AuthorRepository(BaseRepository[Author]):
    """Repositorio para Author - hereda todo de BaseRepository"""

    def __init__(self, db: Session):
        super().__init__(db, Author)
```

¡Eso es todo! `AuthorRepository` ya tiene todos los métodos CRUD.

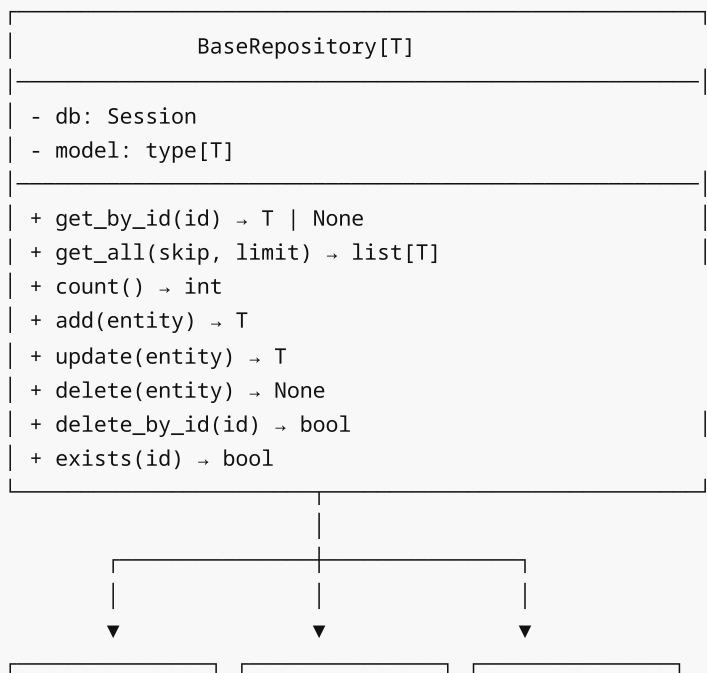
Uso Básico

```
# En un endpoint o service
def get_authors(db: Session = Depends(get_db)):
    repo = AuthorRepository(db)

    # Todos estos métodos vienen de BaseRepository
    authors = repo.get_all(skip=0, limit=10)
    author = repo.get_by_id(1)
    total = repo.count()
    exists = repo.exists(1)

    return authors
```

Diagrama de Herencia



AuthorRepository	PostRepository	TagRepository
[Author]	[Post]	[Tag]
+ get_by_email	+ get_by_tag	+ get_by_slug
+ get_active	+ get_pending	+ get_popular

Métodos Útiles Adicionales

Puedes extender `BaseRepository` con más métodos genéricos:

```
class BaseRepository(Generic[T]):
    # ... métodos anteriores ...

    def get_or_create(
        self,
        defaults: dict,
        **filters
    ) -> tuple[T, bool]:
        """
        Obtiene entidad existente o crea una nueva.

        Returns:
            tuple: (entidad, created: bool)
        """
        stmt = select(self.model).filter_by(**filters)
        entity = self.db.execute(stmt).scalar_one_or_none()

        if entity:
            return entity, False

        # Crear nueva
        entity = self.model(**filters, **defaults)
        self.add(entity)
        return entity, True

    def get_by_ids(self, ids: list[int]) -> list[T]:
        """Obtiene múltiples entidades por sus IDs"""
        if not ids:
            return []
        stmt = select(self.model).where(self.model.id.in_(ids))
        return list(self.db.execute(stmt).scalars().all())

    def filter_by(self, **kwargs) -> list[T]:
        """Filtra por atributos exactos"""
        stmt = select(self.model).filter_by(**kwargs)
        return list(self.db.execute(stmt).scalars().all())

    def first(self, **kwargs) -> T | None:
        """Obtiene primera entidad que coincida"""
        stmt = select(self.model).filter_by(**kwargs).limit(1)
        return self.db.execute(stmt).scalar_one_or_none()
```

⚠ Consideraciones Importantes

1. flush() vs commit()

```
# flush() - Sincroniza con DB pero NO confirma transacción
self.db.flush() # Obtiene IDs, ejecuta queries, pero puede hacer rollback

# commit() - Confirma la transacción (permanente)
self.db.commit() # Ya no se puede deshacer
```

En repositorios usamos `flush()` porque el commit lo maneja la capa superior (Unit of Work o el endpoint).

2. Tipado Correcto

```
# ✅ Correcto - tipo específico en herencia
class AuthorRepository(BaseRepository[Author]):
    pass

# ❌ Incorrecto - sin tipo
class AuthorRepository(BaseRepository):
    pass
```

3. Evitar Lógica de Negocio

```
# ❌ MAL - lógica de negocio en repository
class UserRepository(BaseRepository[User]):
    def create_user(self, data: UserCreate) -> User:
        # ❌ Validación de negocio NO va aquí
        if len(data.password) < 8:
            raise ValueError("Password too short")

        # ❌ Transformación de negocio NO va aquí
        user = User(
            email=data.email,
            password=hash_password(data.password) # ← NO aquí
        )
        return self.add(user)

# ✅ BIEN - repository solo persiste
class UserRepository(BaseRepository[User]):
    # Solo métodos de acceso a datos
    def get_by_email(self, email: str) -> User | None:
        return self.first(email=email)
```

✅ Checklist

- ☐ Entiendo cómo funcionan los Generics en Python
- ☐ Sé la diferencia entre `flush()` y `commit()`
- ☐ Puedo crear un repositorio específico heredando de `BaseRepository`
- ☐ Entiendo que la lógica de negocio NO va en repositorios

Siguiente

Aprenderemos a agregar **métodos específicos** a cada repositorio para queries particulares.

→ [03-repositorios-especificos.md](#)

Repositorios Específicos

Objetivos

- Agregar métodos específicos a repositorios
 - Mantener queries complejas organizadas
 - Usar eager loading desde el repositorio
 - Crear interfaces claras para cada entidad
-

Extendiendo BaseRepository

BaseRepository proporciona operaciones genéricas. Los repositorios específicos agregan **métodos particulares** para cada entidad.

Patrón de Extensión

```
from sqlalchemy import select
from sqlalchemy.orm import Session, selectinload

from models import Author
from repositories.base import BaseRepository

class AuthorRepository(BaseRepository[Author]):
    """Repositorio con métodos específicos para Author"""

    def __init__(self, db: Session):
        super().__init__(db, Author)

    # --- Métodos específicos ---

    def get_by_email(self, email: str) -> Author | None:
        """Busca autor por email único"""
        return self.first(email=email)

    def get_with_posts(self, author_id: int) -> Author | None:
        """Obtiene autor con sus posts cargados (eager loading)"""
        stmt = (
            select(Author)
            .options(selectinload(Author.posts))
            .where(Author.id == author_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def get_active_authors(self) -> list[Author]:
        """Obtiene autores con al menos 1 post"""
        stmt = (
            select(Author)
```

```

        .join(Author.posts)
        .distinct()
    )
    return list(self.db.execute(stmt).scalars().all())

def search_by_name(self, query: str) -> list[Author]:
    """Busca autores por nombre (case-insensitive)"""
    stmt = select(Author).where(
        Author.name.ilike(f"%{query}%")
    )
    return list(self.db.execute(stmt).scalars().all())

```

Ejemplos por Tipo de Entidad

TaskRepository (Proyecto de la semana)

```

from datetime import date
from sqlalchemy import select, and_
from sqlalchemy.orm import Session, joinedload

from models import Task, TaskStatus
from repositories.base import BaseRepository

class TaskRepository(BaseRepository[Task]):
    """Repositorio para operaciones de Task"""

    def __init__(self, db: Session):
        super().__init__(db, Task)

    def get_with_user(self, task_id: int) -> Task | None:
        """Obtiene task con usuario cargado"""
        stmt = (
            select(Task)
            .options(joinedload(Task.user))
            .where(Task.id == task_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def get_by_user(
        self,
        user_id: int,
        status: TaskStatus | None = None
    ) -> list[Task]:
        """Obtiene tasks de un usuario, opcionalmente filtradas por status"""
        stmt = select(Task).where(Task.user_id == user_id)

        if status:
            stmt = stmt.where(Task.status == status)

        stmt = stmt.order_by(Task.due_date.asc().nulls_last())
        return list(self.db.execute(stmt).scalars().all())

    def get_pending(self, user_id: int | None = None) -> list[Task]:

```

```

        """Obtiene tasks pendientes (no completadas)"""
        stmt = select(Task).where(Task.status != TaskStatus.DONE)

        if user_id:
            stmt = stmt.where(Task.user_id == user_id)

        return list(self.db.execute(stmt).scalars().all())

    def get_overdue(self) -> list[Task]:
        """Obtiene tasks vencidas"""
        stmt = select(Task).where(
            and_(
                Task.due_date < date.today(),
                Task.status != TaskStatus.DONE
            )
        )
        return list(self.db.execute(stmt).scalars().all())

    def get_due_today(self) -> list[Task]:
        """Obtiene tasks que vencen hoy"""
        stmt = select(Task).where(Task.due_date == date.today())
        return list(self.db.execute(stmt).scalars().all())

    def count_by_status(self, user_id: int) -> dict[TaskStatus, int]:
        """Cuenta tasks por status para un usuario"""
        from sqlalchemy import func

        stmt = (
            select(Task.status, func.count(Task.id))
            .where(Task.user_id == user_id)
            .group_by(Task.status)
        )
        results = self.db.execute(stmt).all()
        return {status: count for status, count in results}

```

UserRepository

```

from sqlalchemy import select
from sqlalchemy.orm import Session, selectinload

from models import User
from repositories.base import BaseRepository

class UserRepository(BaseRepository[User]):
    """Repositorio para operaciones de User"""

    def __init__(self, db: Session):
        super().__init__(db, User)

    def get_by_email(self, email: str) -> User | None:
        """Busca usuario por email"""
        return self.first(email=email)

    def get_by_username(self, username: str) -> User | None:

```

```

        """Busca usuario por username"""
        return self.first(username=username)

def get_with_tasks(self, user_id: int) -> User | None:
    """Obtiene usuario con todas sus tasks"""
    stmt = (
        select(User)
        .options(selectinload(User.tasks))
        .where(User.id == user_id)
    )
    return self.db.execute(stmt).scalar_one_or_none()

def get_active_users(self) -> list[User]:
    """Obtiene usuarios activos"""
    return self.filter_by(is_active=True)

def email_exists(self, email: str) -> bool:
    """Verifica si el email ya está registrado"""
    return self.get_by_email(email) is not None

```

🎯 Principios para Métodos Específicos

1. Nombres Descriptivos

```

# ✅ BIEN - nombres claros
def get_pending_by_user(self, user_id: int) -> list[Task]: ...
def get_overdue(self) -> list[Task]: ...
def count_by_status(self, user_id: int) -> dict: ...

# ❌ MAL - nombres vagos
def get_tasks(self, user_id: int, status: str) -> list[Task]: ...
def fetch(self) -> list[Task]: ...

```

2. Un Método, Una Responsabilidad

```

# ✅ BIEN - métodos separados
def get_pending(self) -> list[Task]: ...
def get_completed(self) -> list[Task]: ...
def get_overdue(self) -> list[Task]: ...

# ❌ MAL - método con demasiadas opciones
def get_tasks(
    self,
    pending: bool = False,
    completed: bool = False,
    overdue: bool = False,
    user_id: int | None = None,
    ...
) -> list[Task]: ...

```

3. Eager Loading en Repository


```
# ✅ Métodos específicos para cargar relaciones
def get_with_posts(self, author_id: int) -> Author | None:
    """Carga autor CON posts"""
    stmt = select(Author).options(selectinload(Author.posts))...

def get_by_id(self, author_id: int) -> Author | None:
    """Carga autor SIN posts (lazy)"""
    return self.db.get(Author, author_id)
```

4. Retornos Tipados

```
# ✅ BIEN - tipos explícitos
def get_by_email(self, email: str) -> User | None: ...
def get_pending(self) -> list[Task]: ...
def count_by_status(self) -> dict[TaskStatus, int]: ...

# ❌ MAL - sin tipos
def get_by_email(self, email): ...
def get_pending(self): ...
```



Estructura Recomendada

```
repositories/
├── __init__.py
├── base.py           # BaseRepository[T]
├── user_repository.py # UserRepository
├── task_repository.py # TaskRepository
└── tag_repository.py # TagRepository
```

`__init__.py`

```
from repositories.base import BaseRepository
from repositories.user_repository import UserRepository
from repositories.task_repository import TaskRepository

__all__ = [
    "BaseRepository",
    "UserRepository",
    "TaskRepository",
]
```



Queries Complejas

Para queries muy complejas, mantén el código organizado:

```
class TaskRepository(BaseRepository[Task]):

    def get_dashboard_summary(self, user_id: int) -> dict:
        """
        Obtiene resumen para dashboard del usuario.
```

```

Returns:
    {
        "total": int,
        "pending": int,
        "completed": int,
        "overdue": int,
        "due_today": int
    }
"""
from sqlalchemy import func, case

stmt = (
    select(
        func.count(Task.id).label("total"),
        func.sum(case((Task.status == TaskStatus.TODO, 1), else_=0)).label("pending"),
        func.sum(case((Task.status == TaskStatus.DONE, 1), else_=0)).label("completed"),
        func.sum(case(
            (and_(Task.due_date < date.today(), Task.status != TaskStatus.DONE), 1),
            else_=0
        )).label("overdue"),
        func.sum(case((Task.due_date == date.today(), 1), else_=0)).label("due_today"),
    )
    .where(Task.user_id == user_id)
)

result = self.db.execute(stmt).one()

return {
    "total": result.total or 0,
    "pending": result.pending or 0,
    "completed": result.completed or 0,
    "overdue": result.overdue or 0,
    "due_today": result.due_today or 0,
}

```

✓ Checklist

- ☐ Sé cuándo agregar métodos específicos vs usar BaseRepository
- ☐ Uso nombres descriptivos para métodos
- ☐ Implemento eager loading en métodos que lo necesitan
- ☐ Mantengo los repositorios enfocados en acceso a datos

Siguiente

Aprenderemos el patrón **Unit of Work** para manejar transacciones de forma coordinada.

→ [04-unit-of-work.md](#)

Unit of Work Pattern

 Unit of Work

🎯 Objetivos

- Entender el patrón Unit of Work
- Coordinar transacciones entre múltiples repositorios
- Implementar commit/rollback centralizado
- Integrar con FastAPI dependencies

🔍 El Problema: Transacciones Distribuidas

Cuando usas múltiples repositorios, ¿quién hace commit?

```
# ❌ Problema: múltiples commits
class OrderService:
    def create_order(self, data: OrderCreate) -> Order:
        # Crear orden
        order = Order(...)
        self.order_repo.add(order)
        # ¿Commit aquí?

        # Actualizar inventario
        for item in data.items:
            product = self.product_repo.get_by_id(item.product_id)
            product.stock -= item.quantity
            self.product_repo.update(product)
            # ¿Commit aquí?

        # Si falla el inventario, la orden ya está guardada ❌
```

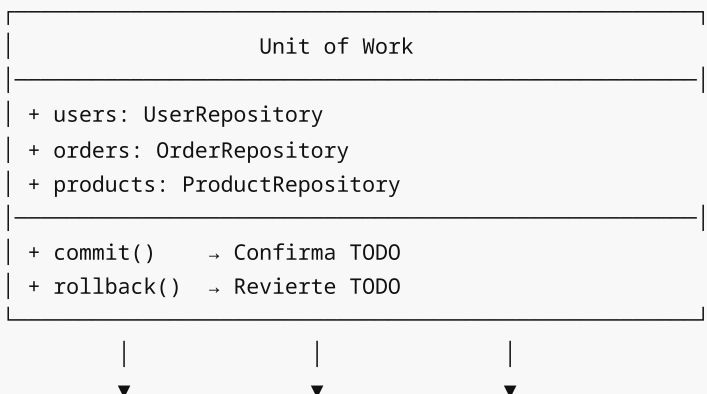
Escenario Problemático

1. Crear orden → commit ✓
2. Actualizar producto 1 → commit ✓
3. Actualizar producto 2 → ❌ ERROR (sin stock)

Resultado: Orden creada, producto 1 actualizado, producto 2 sin cambios
Estado inconsistente de la base de datos

✅ Solución: Unit of Work

El **Unit of Work** (UoW) coordina los cambios de múltiples repositorios en una sola transacción:



users
table

orders
table

products
table

Implementación Básica

```
# repositories/unit_of_work.py
from sqlalchemy.orm import Session

from database import SessionLocal
from repositories import UserRepository, TaskRepository

class UnitOfWork:
    """
    Coordina transacciones entre repositorios.

    Uso:
        with UnitOfWork() as uow:
            user = uow.users.get_by_id(1)
            task = Task(title="New", user_id=user.id)
            uow.tasks.add(task)
            uow.commit()
    """

    def __init__(self, session: Session | None = None):
        # Usa sesión existente o crea nueva
        self._session = session or SessionLocal()
        self._owns_session = session is None

    def __enter__(self) -> "UnitOfWork":
        # Crear repositorios con la misma sesión
        self.users = UserRepository(self._session)
        self.tasks = TaskRepository(self._session)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            # Hubo excepción → rollback
            self.rollback()

        if self._owns_session:
            self._session.close()

    def commit(self) -> None:
        """Confirma todos los cambios"""
        self._session.commit()

    def rollback(self) -> None:
        """Revierte todos los cambios"""
        self._session.rollback()

    @property
    def session(self) -> Session:
```

```
"""Expone sesión para casos especiales"""  
return self._session
```

Uso con Context Manager

Caso Simple

```
# Crear usuario y task en una transacción
with UnitOfWork() as uow:
    user = User(name="John", email="john@example.com")
    uow.users.add(user)

    task = Task(title="First task", user_id=user.id)
    uow.tasks.add(task)

    uow.commit() # Ambos se guardan o ninguno
```

Manejo de Errores

```
with UnitOfWork() as uow:
    try:
        user = uow.users.get_by_id(1)
        if not user:
            raise NotFoundError("User not found")

        task = Task(title="New task", user_id=user.id)
        uow.tasks.add(task)

        uow.commit()

    except Exception:
        uow.rollback()
        raise
```

Integración con FastAPI

Opción 1: Dependency que crea UoW

```
# dependencies.py
from repositories.unit_of_work import UnitOfWork

def get_uow():
    """Dependency que proporciona Unit of Work"""
    with UnitOfWork() as uow:
        yield uow

# routers/tasks.py
@router.post("/", response_model=TaskResponse)
def create_task(
```

```

task_data: TaskCreate,
uow: UnitOfWork = Depends(get_uow)
):
    # Verificar usuario
    user = uow.users.get_by_id(task_data.user_id)
    if not user:
        raise HTTPException(404, "User not found")

    # Crear task
    task = Task(**task_data.model_dump())
    uow.tasks.add(task)
    uow.commit()

    return task

```

Opción 2: UoW recibe Session existente

```

# dependencies.py
from database import get_db

def get_uow(db: Session = Depends(get_db)):
    """UoW con sesión de FastAPI"""
    with UnitOfWork(session=db) as uow:
        yield uow

```

UoW en Services

El patrón más limpio es usar UoW dentro de los services:

```

# services/task_service.py
from repositories.unit_of_work import UnitOfWork
from exceptions import NotFoundError

class TaskService:
    """Service que usa Unit of Work internamente"""

    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_task(self, data: TaskCreate) -> Task:
        # Validar usuario existe
        user = self.uow.users.get_by_id(data.user_id)
        if not user:
            raise NotFoundError(f"User {data.user_id} not found")

        # Crear task
        task = Task(**data.model_dump())
        self.uow.tasks.add(task)

        # NO commit aquí - lo hace quien llama al service
        return task

```

```

def complete_task(self, task_id: int) -> Task:
    task = self.uow.tasks.get_by_id(task_id)
    if not task:
        raise NotFoundError(f"Task {task_id} not found")

    task.status = TaskStatus.DONE
    task.completed_at = datetime.utcnow()

    return task

def assign_task(self, task_id: int, user_id: int) -> Task:
    task = self.uow.tasks.get_by_id(task_id)
    if not task:
        raise NotFoundError(f"Task {task_id} not found")

    user = self.uow.users.get_by_id(user_id)
    if not user:
        raise NotFoundError(f"User {user_id} not found")

    task.user_id = user_id
    return task

```

Uso en Router

```

# routers/tasks.py
@router.post("/", response_model=TaskResponse)
def create_task(
    task_data: TaskCreate,
    uow: UnitOfWork = Depends(get_uow)
):
    service = TaskService(uow)

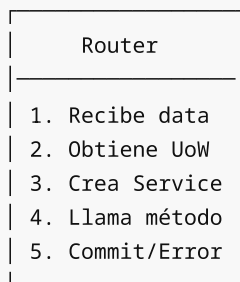
    try:
        task = service.create_task(task_data)
        uow.commit()
        return task
    except NotFoundError as e:
        raise HTTPException(404, str(e))

```

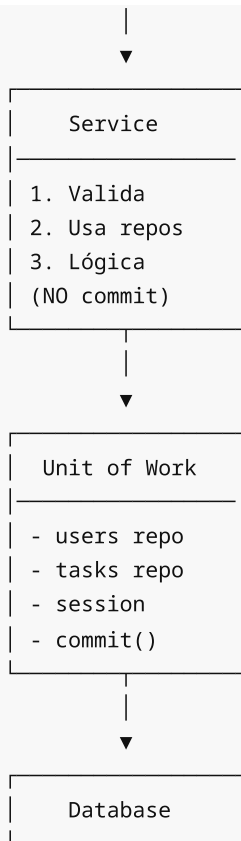
Flujo Completo

Request POST /tasks

|



|



⚠ Consideraciones

¿Cuándo Usar UoW?

Situación	¿UoW?
Operación con un solo repositorio	Opcional
Operación con múltiples repositorios	✅ Recomendado
Transacciones que deben ser atómicas	✅ Necesario
CRUD simple	Probablemente no

Errores Comunes

```
# ❌ MAL - commit en cada operación
def create_order(self, data):
    order = Order(...)
    self.uow.orders.add(order)
    self.uow.commit() # ← Commit prematuro

    for item in data.items:
        # Si falla aquí, la orden ya está guardada
        ...

# ✅ BIEN - commit al final
def create_order(self, data):
    order = Order(...)
    self.uow.orders.add(order)
```



```
for item in data.items:
    ...

# Commit después de todas las operaciones
self.uow.commit()
```

✓ Checklist

- ☐ Entiendo qué problema resuelve Unit of Work
- ☐ Sé implementar UoW como context manager
- ☐ Puedo integrar UoW con FastAPI dependencies
- ☐ Sé cuándo hacer commit (al final, no en medio)

🔗 Siguiente

Aprenderemos cómo el Repository Pattern **facilita el testing** con mocks y fakes.

→ [05-testing-con-repositories.md](#)

📖 Testing con Repositories

🎯 Objetivos

- Entender por qué Repository facilita el testing
- Crear Fake Repositories para tests
- Testear Services sin base de datos
- Aplicar inyección de dependencias para tests

🔍 El Problema: Tests con Base de Datos

Sin Repository Pattern, testear services requiere una BD real:

```
# ❌ Test que depende de base de datos
def test_create_task():
    # Necesita BD configurada
    db = SessionLocal()

    # Necesita datos previos
    user = User(name="Test", email="test@test.com")
    db.add(user)
    db.commit()

    # Test real
    service = TaskService(db)
    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"

    # Limpiar
    db.delete(task)
    db.delete(user)
```

```
db.commit()
db.close()
```

Problemas

Problema	Impacto
Lento	BD es I/O, tests lentos
Frágil	Depende de estado de BD
Complejo	Setup y teardown
No aislado	Tests pueden interferir

✓ Solución: Fake Repositories

Un **Fake Repository** simula el comportamiento del repositorio real usando estructuras en memoria:

```
# tests/fakes.py
from models import Task, User

class FakeTaskRepository:
    """Repositorio falso que usa diccionario en memoria"""

    def __init__(self):
        self.tasks: dict[int, Task] = {}
        self._next_id = 1

    def get_by_id(self, task_id: int) -> Task | None:
        return self.tasks.get(task_id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Task]:
        tasks = list(self.tasks.values())
        return tasks[skip:skip + limit]

    def add(self, task: Task) -> Task:
        task.id = self._next_id
        self.tasks[task.id] = task
        self._next_id += 1
        return task

    def update(self, task: Task) -> Task:
        self.tasks[task.id] = task
        return task

    def delete(self, task: Task) -> None:
        if task.id in self.tasks:
            del self.tasks[task.id]

    # Métodos específicos
    def get_by_user(self, user_id: int) -> list[Task]:
        return [t for t in self.tasks.values() if t.user_id == user_id]

    def get_pending(self) -> list[Task]:
```

```
return [t for t in self.tasks.values() if t.status != TaskStatus.DONE]
```

```
class FakeUserRepository:
    """Repositorio falso para User"""

    def __init__(self):
        self.users: dict[int, User] = {}
        self._next_id = 1

    def get_by_id(self, user_id: int) -> User | None:
        return self.users.get(user_id)

    def get_by_email(self, email: str) -> User | None:
        for user in self.users.values():
            if user.email == email:
                return user
        return None

    def add(self, user: User) -> User:
        user.id = self._next_id
        self.users[user.id] = user
        self._next_id += 1
        return user
```

Tests con Fakes

Test de Service

```
# tests/test_task_service.py
import pytest
from datetime import date

from models import User, Task, TaskStatus
from schemas import TaskCreate
from services import TaskService
from exceptions import NotFoundError
from tests.fakes import FakeTaskRepository, FakeUserRepository

class TestTaskService:
    """Tests para TaskService usando fakes"""

    def setup_method(self):
        """Setup antes de cada test"""
        self.task_repo = FakeTaskRepository()
        self.user_repo = FakeUserRepository()

        # Usuario de prueba
        self.test_user = User(name="John", email="john@test.com")
        self.user_repo.add(self.test_user)

        # Service con fakes
        self.service = TaskService(
```

```

        task_repo=self.task_repo,
        user_repo=self.user_repo
    )

def test_create_task_success(self):
    """Crear task con usuario válido"""
    data = TaskCreate(
        title="Test Task",
        description="Description",
        user_id=self.test_user.id
    )

    task = self.service.create_task(data)

    assert task.id == 1
    assert task.title == "Test Task"
    assert task.user_id == self.test_user.id
    assert task.status == TaskStatus.TODO

def test_create_task_user_not_found(self):
    """Error si usuario no existe"""
    data = TaskCreate(
        title="Test",
        user_id=999 # No existe
    )

    with pytest.raises(NotFoundError) as exc:
        self.service.create_task(data)

    assert "User 999 not found" in str(exc.value)

def test_complete_task_success(self):
    """Completar task existente"""
    # Crear task primero
    task = Task(title="Test", user_id=self.test_user.id)
    self.task_repo.add(task)

    completed = self.service.complete_task(task.id)

    assert completed.status == TaskStatus.DONE
    assert completed.completed_at is not None

def test_complete_task_not_found(self):
    """Error si task no existe"""
    with pytest.raises(NotFoundError):
        self.service.complete_task(999)

def test_complete_task_already_done(self):
    """Error si task ya está completada"""
    task = Task(
        title="Done",
        user_id=self.test_user.id,
        status=TaskStatus.DONE
    )
    self.task_repo.add(task)

```

```

        with pytest.raises(BusinessError) as exc:
            self.service.complete_task(task.id)

        assert "already completed" in str(exc.value)

    def test_get_user_tasks(self):
        """Obtener tasks de un usuario"""
        # Crear varias tasks
        for i in range(3):
            task = Task(title=f"Task {i}", user_id=self.test_user.id)
            self.task_repo.add(task)

        tasks = self.service.get_user_tasks(self.test_user.id)

        assert len(tasks) == 3

```

Fake Unit of Work

Para tests más completos, crea un Fake UoW:

```

# tests/fakes.py
class FakeUnitOfWork:
    """Unit of Work falso para tests"""

    def __init__(self):
        self.users = FakeUserRepository()
        self.tasks = FakeTaskRepository()
        self.committed = False
        self.rolled_back = False

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.rollback()

    def commit(self):
        self.committed = True

    def rollback(self):
        self.rolled_back = True

# Test usando Fake UoW
def test_transaction_commits():
    uow = FakeUnitOfWork()

    user = User(name="Test", email="test@test.com")
    uow.users.add(user)

    task = Task(title="Task", user_id=user.id)
    uow.tasks.add(task)

    uow.commit()

```

```
assert uow.committed is True
assert len(uow.tasks.tasks) == 1
```

Comparación: Con y Sin Fakes

Sin Fakes (Integration Test)

```
# Lento, requiere BD, complejo setup
@pytest.fixture
def db():
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    Session = sessionmaker(bind=engine)
    session = Session()
    yield session
    session.close()

def test_create_task(db):
    # Setup en BD real
    user = User(name="Test", email="test@test.com")
    db.add(user)
    db.commit()

    repo = TaskRepository(db)
    service = TaskService(task_repo=repo, user_repo=UserRepository(db))

    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"
```

Con Fakes (Unit Test)

```
# Rápido, sin BD, simple
def test_create_task():
    task_repo = FakeTaskRepository()
    user_repo = FakeUserRepository()

    user = User(name="Test", email="test@test.com")
    user_repo.add(user)

    service = TaskService(task_repo=task_repo, user_repo=user_repo)

    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"
```

Ejecutar Tests

```
# Ejecutar todos los tests
pytest tests/
```

```
# Ejecutar con verbose
pytest tests/ -v

# Ejecutar test específico
pytest tests/test_task_service.py::TestTaskService::test_create_task_success

# Ver cobertura
pytest tests/ --cov=services --cov-report=html
```

Estructura de Tests

```
tests/
├── __init__.py
├── conftest.py          # Fixtures compartidos
├── fakes.py            # Fake repositories
├── test_task_service.py # Tests de TaskService
├── test_user_service.py # Tests de UserService
├── integration/        # Tests de integración (con BD)
│   └── test_api.py
```

conftest.py

```
import pytest
from tests.fakes import FakeTaskRepository, FakeUserRepository, FakeUnitOfWork

@pytest.fixture
def task_repo():
    return FakeTaskRepository()

@pytest.fixture
def user_repo():
    return FakeUserRepository()

@pytest.fixture
def uow():
    return FakeUnitOfWork()

@pytest.fixture
def test_user(user_repo):
    from models import User
    user = User(name="Test User", email="test@example.com")
    return user_repo.add(user)
```

Beneficios del Testing con Repositories

Beneficio	Descripción
-----------	-------------

Velocidad	Tests en memoria son instantáneos
Aislamiento	Cada test tiene su propio estado
Simplicidad	No hay setup de BD
Confiabilidad	No depende de infraestructura
Enfoque	Testa lógica de negocio pura

✓ Checklist

- ☐ Sé crear Fake Repositories para tests
- ☐ Puedo testear Services sin base de datos
- ☐ Entiendo la diferencia entre unit test e integration test
- ☐ Sé estructurar tests con pytest

🎯 Resumen de la Semana

Esta semana aprendiste:

1. **Repository Pattern**: Abstrae acceso a datos
2. **BaseRepository**: Código genérico reutilizable
3. **Repositorios Específicos**: Queries particulares
4. **Unit of Work**: Transacciones coordinadas
5. **Testing**: Fake repositories para tests unitarios

La arquitectura actual:

```
Router → Service → Repository → Database
      ↓
      Unit of Work (coordina transacciones)
```

→ Siguiente semana: **MVC Completo** con todas las capas integradas

🔨 Práctica 01: Primer Repositorio

🎯 Objetivo

Crear tu primer repositorio separando el acceso a datos del service.

📋 Contexto

Partimos de un service que accede directamente a SQLAlchemy (como en Week-06) y lo refactorizamos para usar un repositorio.

📝 Instrucciones

Paso 1: Revisar el código inicial

Abre `starter/main.py` y observa cómo el service accede directamente a la base de datos.

Paso 2: Crear el repositorio

En `starter/repositories.py`, descomenta el código del `ProductRepository`.

Paso 3: Refactorizar el service

En `starter/services.py`, descomenta el código que usa el repositorio en lugar de acceder directamente a SQLAlchemy.

Paso 4: Actualizar el endpoint

En `starter/main.py`, descomenta el código que inyecta el repositorio.

Paso 5: Probar

```
cd starter
uv run fastapi dev main.py
```

Visita <http://localhost:8000/docs> y prueba los endpoints.

✓ Resultado Esperado

- Service NO tiene imports de SQLAlchemy
- Repository maneja todas las operaciones de BD
- Endpoints funcionan igual que antes

Archivos

- `starter/models.py` - Modelo Product
- `starter/repositories.py` - ProductRepository (descomentar)
- `starter/services.py` - ProductService refactorizado
- `starter/main.py` - Endpoints actualizados

Práctica 02: Repositorio Genérico

Objetivo

Implementar un `BaseRepository` genérico para evitar duplicación de código CRUD.

Contexto

Cuando tienes múltiples entidades, el código CRUD se repite. Un repositorio genérico con Python Generics resuelve esto.

Instrucciones

Paso 1: Entender el problema

Revisa `starter/repositories_before.py` para ver la duplicación de código.

Paso 2: Crear BaseRepository

En `starter/base_repository.py`, descomenta la implementación de `BaseRepository`.

Paso 3: Crear repositorios específicos

En `starter/repositories.py`, descomenta los repositorios que heredan de `BaseRepository`.

Paso 4: Probar

```
cd starter
uv run python main.py
```

✓ Resultado Esperado

- `BaseRepository` con métodos genéricos CRUD
- `ProductRepository` y `CategoryRepository` heredando
- Métodos específicos en cada repositorio
- Sin duplicación de código

🔨 Práctica 03: Integración Service-Repository

🎯 Objetivo

Conectar Services con Repositories de forma correcta, manteniendo la separación de responsabilidades.

📋 Contexto

El Service debe recibir los Repositories por inyección de dependencias, no crearlos internamente.

📝 Instrucciones

Paso 1: Revisar modelos y repositorios

Los archivos `models.py` y `repositories.py` ya están completos.

Paso 2: Implementar OrderService

En `starter/services.py`, descomenta el `OrderService` que usa repositorios.

Paso 3: Configurar dependencias FastAPI

En `starter/main.py`, descomenta las funciones de dependencias y endpoints.

Paso 4: Probar

```
cd starter
uv run fastapi dev main.py
```

Prueba crear usuarios, productos y órdenes en `/docs`.

✓ Resultado Esperado

- Service NO conoce SQLAlchemy
- Repositories son inyectados via `Depends()`
- Lógica de negocio (validaciones) en Service
- Acceso a datos en Repository

🔨 Práctica 04: Unit of Work

Objetivo

Implementar el patrón Unit of Work para coordinar transacciones entre repositorios.

Contexto

Cuando una operación involucra múltiples repositorios, necesitamos que todos los cambios sean atómicos (todo o nada).

Instrucciones

Paso 1: Revisar el problema

En `starter/problem.py` hay un ejemplo de transacción sin Unit of Work.

Paso 2: Implementar UnitOfWork

En `starter/unit_of_work.py`, descomenta la clase `UnitOfWork`.

Paso 3: Usar UoW en Service

En `starter/services.py`, descomenta el servicio que usa UoW.

Paso 4: Probar

```
cd starter
uv run python main.py
```

Resultado Esperado

- Unit of Work coordina múltiples repositorios
 - Una sola sesión compartida
 - Commit/rollback atómico
 - Context manager para cleanup automático
-

Proyecto Semana 07: API con Repository Pattern

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

 **IMPORTANTE:** Cada aprendiz trabaja sobre un dominio diferente.

Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que **NO** está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Entity	Item	{YourEntity}
Repository	IItemRepository	I{YourEntity}Repository
Fake Repo	FakeItemRepository	Fake{YourEntity}Repository

Objetivo

Implementar el **Repository Pattern** con:

- Interfaces (Protocols) para repositorios
 - Implementaciones SQLAlchemy
 - Fake repositories para testing
 - Inyección de dependencias
-

Requisitos Funcionales (Adapta a tu Dominio)

Repository Interface

```
# Ejemplo genérico (Warehouse)
from typing import Protocol

class IItemRepository(Protocol):
    """Interfaz del repositorio de items"""

    async def get_by_id(self, id: int) -> Item | None: ...
    async def get_by_sku(self, sku: str) -> Item | None: ...
    async def list_all(self, skip: int, limit: int) -> list[Item]: ...
    async def create(self, item: ItemCreate) -> Item: ...
    async def update(self, id: int, data: ItemUpdate) -> Item | None: ...
    async def delete(self, id: int) -> bool: ...
    async def count(self) -> int: ...
```

SQLAlchemy Implementation

```
# Ejemplo genérico
class SQLAlchemyItemRepository:
    def __init__(self, session: AsyncSession):
        self._session = session

    async def get_by_id(self, id: int) -> Item | None:
        result = await self._session.execute(
            select(ItemModel).where(ItemModel.id == id)
        )
        return result.scalar_one_or_none()

    async def get_by_sku(self, sku: str) -> Item | None:
        result = await self._session.execute(
            select(ItemModel).where(ItemModel.sku == sku)
        )
        return result.scalar_one_or_none()
```

Fake Repository (Testing)

```
# Ejemplo genérico
class FakeItemRepository:
    def __init__(self):
        self._items: dict[int, Item] = {}
```

```

        self._next_id = 1

    async def get_by_id(self, id: int) -> Item | None:
        return self._items.get(id)

    async def create(self, item: ItemCreate) -> Item:
        new_item = Item(id=self._next_id, **item.model_dump())
        self._items[self._next_id] = new_item
        self._next_id += 1
        return new_item

```

Dependency Injection

```

# Ejemplo genérico
def get_item_repository(
    session: AsyncSession = Depends(get_session)
) -> IItemRepository:
    return SQLAlchemyItemRepository(session)

@router.get("/items/{id}")
async def get_item(
    id: int,
    repo: IItemRepository = Depends(get_item_repository)
):
    item = await repo.get_by_id(id)
    if not item:
        raise HTTPException(404, "Item not found")
    return item

```

Estructura del Proyecto

```

starter/
├─ main.py
├─ domain/
│   └─ entities.py
├─ repositories/
│   ├── interfaces.py
│   ├── sqlalchemy_repo.py
│   └─ fake_repo.py
├─ services/
├─ routers/
├─ dependencies.py
├─ tests/
│   └─ test_with_fake_repo.py
├─ pyproject.toml
├─ Dockerfile
└─ docker-compose.yml

```

Criterios de Evaluación

Criterio	Puntos
----------	--------

Funcionalidad (40%)	
Interface bien definida	15
SQLAlchemy repo funciona	15
Fake repo para tests	10
Adaptación al Dominio (35%)	
Métodos específicos del negocio	12
Queries coherentes con dominio	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Inyección de dependencias	10
Tests con fake repo	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ❌ **No copies** el ejemplo genérico "ItemRepository"
- ✅ **Diseña** métodos específicos de tu dominio
- ✅ **Implementa** queries relevantes para tu negocio

📚 Recursos

- [Repository Pattern](#)
- [Python Protocol](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)

📖 Glosario - Semana 07

Repository Pattern y Unit of Work

A

Abstract Repository

Interfaz o clase base que define el contrato que deben cumplir los repositorios concretos. Permite intercambiar implementaciones (DB real, fake, etc.).

Aggregate

En Domain-Driven Design, un cluster de objetos de dominio que se tratan como una unidad. El Repository típicamente opera sobre aggregates.

B

BaseRepository

Clase genérica que implementa operaciones CRUD comunes. Usa `Generic[T]` para trabajar con cualquier tipo de entidad.

```
class BaseRepository(Generic[T]):
    def get_by_id(self, id: int) -> T | None: ...
    def add(self, entity: T) -> T: ...
```

Bounded Context

Límite conceptual donde un modelo de dominio particular es definido y aplicable. Los repositorios pertenecen a un bounded context específico.

C

CRUD

Create, Read, Update, Delete - Las cuatro operaciones básicas de persistencia que un repositorio típicamente implementa.

Commit

Operación que confirma todos los cambios pendientes en una transacción de base de datos, haciéndolos permanentes.

```
session.commit() # Guarda cambios permanentemente
```

D

Data Access Layer (DAL)

Capa de la aplicación responsable de la comunicación con la base de datos. Los repositorios forman parte de esta capa.

Dependency Injection (DI)

Patrón donde las dependencias se pasan a un objeto en lugar de que el objeto las cree. Los services reciben repositorios por inyección.

```
class UserService:
    def __init__(self, repo: UserRepository):
        self.repo = repo # Inyectado, no creado
```

F

Fake Repository

Implementación de repositorio que usa estructuras en memoria (dict, list) en lugar de base de datos real. Útil para testing.

```
class FakeUserRepository:
    def __init__(self):
```

```
self._data: dict[int, User] = {}
```

Flush

Operación que sincroniza los cambios pendientes con la base de datos sin hacer commit. Permite obtener IDs generados.

```
session.add(entity)
session.flush() # Sincroniza, obtiene ID
# entity.id ahora tiene valor
session.commit() # Confirma transacción
```

G

Generic

En Python, clase que puede trabajar con diferentes tipos mediante parámetros de tipo. Se define usando `Generic[T]` .

```
from typing import TypeVar, Generic

T = TypeVar("T")

class Repository(Generic[T]):
    def get(self, id: int) -> T: ...
```

I

Identity Map

Patrón que asegura que cada objeto se carga solo una vez por sesión. SQLAlchemy Session implementa este patrón automáticamente.

Inversion of Control (IoC)

Principio donde el control del flujo se invierte: en lugar de que el código llame a bibliotecas, el framework llama al código del usuario.

L

Lazy Loading

Técnica donde los datos relacionados se cargan solo cuando se acceden, no al cargar el objeto principal.

M

Mock

Objeto que simula el comportamiento de objetos reales de manera controlada. Diferente de Fake: los mocks verifican interacciones.

```
from unittest.mock import Mock

mock_repo = Mock()
mock_repo.get_by_id.return_value = user
```


P

Persistence Ignorance

Principio donde las entidades de dominio no conocen cómo se persisten. El repositorio encapsula esa lógica.

Port

En arquitectura hexagonal, interfaz que define cómo el dominio se comunica con el exterior. Un repositorio es un "port" de salida.

R

Repository

Patrón que encapsula la lógica de acceso a datos, proporcionando una interfaz de colección para acceder a objetos del dominio.

```
class UserRepository:
    def get_by_id(self, id: int) -> User | None: ...
    def get_by_email(self, email: str) -> User | None: ...
    def add(self, user: User) -> User: ...
```

Rollback

Operación que revierte todos los cambios pendientes en una transacción, restaurando el estado anterior.

```
try:
    # operaciones...
    session.commit()
except:
    session.rollback() # Revierte cambios
```

S

Session

En SQLAlchemy, objeto que gestiona las operaciones de persistencia. Implementa Identity Map y Unit of Work.

Service Layer

Capa que contiene la lógica de negocio de la aplicación. Los services usan repositorios para acceder a datos.

```
class UserService:
    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_user(self, data: UserCreate) -> User:
        # Lógica de negocio aquí
        return self.uow.users.add(user)
```

T

Transaction

Secuencia de operaciones de base de datos que se ejecutan como una unidad atómica. O todas se completan o ninguna.

TypeVar

En Python, variable de tipo que representa un tipo genérico. Se usa con `Generic` para crear clases genéricas.

```
from typing import TypeVar

T = TypeVar("T", bound=Base) # T debe heredar de Base
```

U

Unit of Work (UoW)

Patrón que mantiene una lista de objetos afectados por una transacción y coordina la escritura de cambios.

```
class UnitOfWork:
    def __enter__(self):
        self.users = UserRepository(self.session)
        self.tasks = TaskRepository(self.session)
        return self

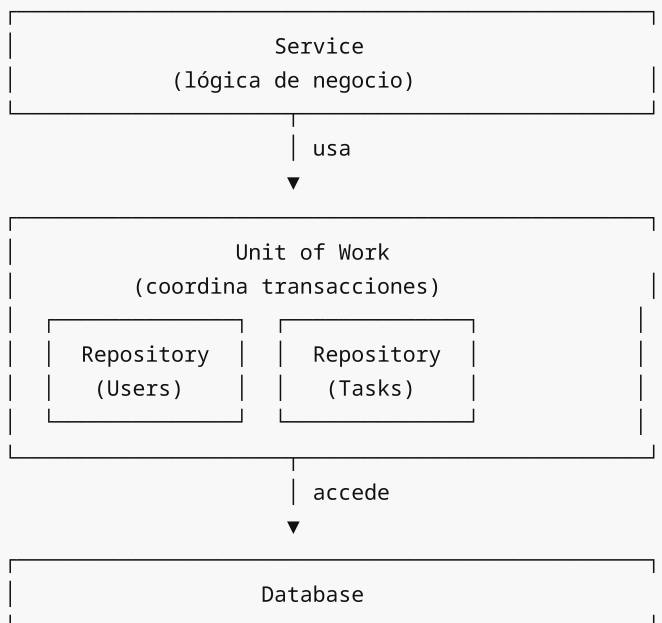
    def commit(self):
        self.session.commit()

    def rollback(self):
        self.session.rollback()
```

Unit Test

Test que verifica una unidad aislada de código (función, clase). Los fake repositories permiten tests unitarios de services.

Diagrama de Relaciones



Referencias

- Fowler, M. - Patterns of Enterprise Application Architecture
- Evans, E. - Domain-Driven Design
- Percival, H. & Gregory, B. - Architecture Patterns with Python

Semana 07: Repository Pattern

Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

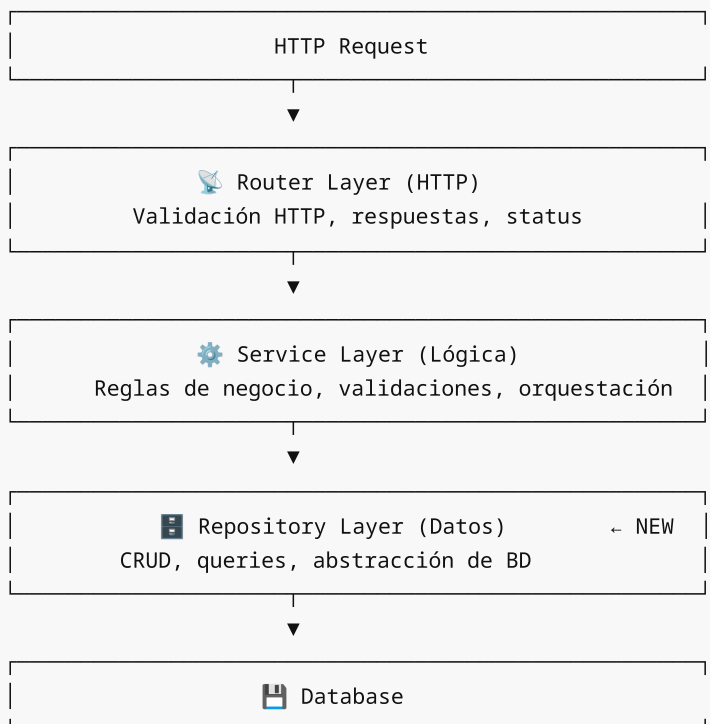
-  Entender el **Repository Pattern** y su propósito
-  Separar el **acceso a datos** de la **lógica de negocio**
-  Implementar **repositorios genéricos** reutilizables
-  Aplicar el patrón **Unit of Work** para transacciones
-  Facilitar el **testing** con repositorios mock
-  Evolucionar la arquitectura: Router → Service → **Repository**

Contexto Arquitectónico

Evolución del Bootcamp

Semana 05: Monolítico → Todo en endpoints
Semana 06: + Service Layer → Lógica separada
Semana 07: + Repository → Acceso a datos separado ← ESTAMOS AQUÍ
Semana 08: MVC Completo → Arquitectura por capas

Arquitectura Actual (Semana 07)



Requisitos Previos

Antes de comenzar, asegúrate de:

- ✓ Haber completado la **Semana 06** (Service Layer)
- ✓ Entender relaciones SQLAlchemy (1:N, N:M)
- ✓ Conocer el patrón Service Layer
- ✓ Saber usar `Depends()` de FastAPI

Estructura de la Semana

```
week-07/
├── README.md                      # Este archivo
├── rubrica-evaluacion.md          # Criterios de evaluación
├── 0-assets/                     # Diagramas SVG
│   ├── 01-repository-pattern.svg
│   ├── 02-capas-arquitectura.svg
│   └── 03-unit-of-work.svg
├── 1-teoria/
│   ├── 01-introduccion-repository-pattern.md
│   ├── 02-repositorio-generico.md
│   ├── 03-repositorios-especificos.md
│   ├── 04-unit-of-work.md
│   └── 05-testing-con-repositories.md
├── 2-practicas/
│   ├── 01-primer-repositorio/
│   ├── 02-repositorio-generico/
│   ├── 03-integracion-service-repository/
│   └── 04-unit-of-work/
├── 3-proyecto/
│   ├── README.md
│   ├── starter/                  # Código inicial
│   └── solution/                 # Solución (oculta)
├── 4-recursos/
│   └── README.md
└── 5-glosario/
    └── README.md
```

Contenidos

1 Teoría

Archivo	Tema	Duración
01-introduccion-repository-pattern.md	Qué es y por qué usar Repository	20 min
02-repositorio-generico.md	Implementación de BaseRepository	25 min
03-repositorios-especificos.md	Repositorios con métodos custom	20 min
04-unit-of-work.md	Patrón Unit of Work	25 min
05-testing-con-repositories.md	Testing con mocks y fakes	20 min

2 Prácticas

Práctica	Tema	Duración
01-primer-repositorio	Crear repositorio básico	30 min
02-repositorio-generico	Implementar BaseRepository	35 min
03-integracion-service-repository	Conectar Service con Repository	30 min
04-unit-of-work	Implementar Unit of Work	35 min

3 Proyecto

Proyecto	Descripción	Duración
Task Manager API	API de gestión de tareas con Repository Pattern	2 horas

Distribución del Tiempo (6 horas)

Actividad	Tiempo
 Teoría	1.5 h
 Prácticas	2.5 h
 Proyecto	2 h

Competencias a Desarrollar

Técnicas

- Implementar el patrón Repository en Python
- Crear repositorios genéricos con tipado estricto
- Aplicar Unit of Work para transacciones
- Escribir tests con repositorios mock

Arquitectónicas

- Separar responsabilidades por capas
- Diseñar interfaces para abstracción
- Facilitar el testing mediante inyección de dependencias

Entregable

Proyecto: [Task Manager](#)

API de gestión de tareas funcionando con:

- ☐ BaseRepository genérico
- ☐ TaskRepository y UserRepository
- ☐ TaskService usando repositorios
- ☐ Tests unitarios básicos

Navegación

← Anterior	Inicio	Siguiente →
----------------------------	------------------------	-----------------------------

Tip de la Semana

"Un repositorio es como un bibliotecario": no necesitas saber cómo está organizada la biblioteca internamente, solo le pides el libro que necesitas y él sabe dónde encontrarlo.

Referencias Rápidas

- [Repository Pattern - Martin Fowler](#)
- [SQLAlchemy ORM Tutorial](#)
- [FastAPI Dependencies](#)

Rúbrica de Evaluación - Semana 07

Repository Pattern

Competencias Evaluadas

Competencia	Peso
Implementación de Repository Pattern	35%
Repositorio Genérico (BaseRepository)	25%
Integración Service-Repository	25%
Testing con Repositories	15%

Criterios de Evaluación

1. Implementación de Repository Pattern (35%)

Excelente (100%)

-  Repositorio abstrae completamente el acceso a datos
-  Service no tiene imports de SQLAlchemy
-  Métodos CRUD implementados correctamente
-  Manejo de sesiones apropiado

Bueno (75%)

-  Repositorio funcional con métodos básicos
-  alguna dependencia de SQLAlchemy en services
-  CRUD funciona correctamente

Suficiente (50%)

-  Repositorio implementado básicamente
-  Separación de capas incompleta
-  Algunos métodos faltantes

Insuficiente (< 50%)

-  No hay separación clara
-  Lógica de datos mezclada con negocio

2. Repositorio Genérico (25%)

Excelente (100%)

```
# Implementación esperada
class BaseRepository(Generic[T]):
    def __init__(self, db: Session, model: type[T]):
        self.db = db
        self.model = model

    def get_by_id(self, id: int) -> T | None: ...
    def get_all(self, skip: int, limit: int) -> list[T]: ...
    def create(self, obj: T) -> T: ...
    def update(self, obj: T) -> T: ...
    def delete(self, id: int) -> bool: ...
```

-  Uso correcto de Generics
-  Type hints completos
-  Métodos reutilizables
-  Repositorios específicos heredan correctamente

Bueno (75%)

-  BaseRepository funcional
-  Type hints incompletos
-  Herencia implementada

Suficiente (50%)

-  Clase base existe
-  Sin generics
-  Duplicación de código en repositorios

Insuficiente (< 50%)

-  Sin repositorio genérico
 -  Código duplicado en cada repositorio
-

3. Integración Service-Repository (25%)

Excelente (100%)

```
# Service usando Repository
class TaskService:
    def __init__(self, task_repo: TaskRepository, user_repo: UserRepository):
        self.task_repo = task_repo
        self.user_repo = user_repo

    def create_task(self, data: TaskCreate) -> Task:
        # Validación de negocio
        user = self.user_repo.get_by_id(data.user_id)
        if not user:
            raise NotFoundError("User not found")

        # Delegación a repository
        task = Task(**data.model_dump())
        return self.task_repo.create(task)
```

-  Service recibe repositorios por inyección
-  Service NO conoce SQLAlchemy
-  Lógica de negocio en Service
-  Acceso a datos en Repository

Bueno (75%)

-  Integración funcional
-  Algo de lógica de datos en service

Suficiente (50%)

-  Service usa repository
-  Mezcla de responsabilidades

Insuficiente (< 50%)

-  Service accede directamente a DB
-  No usa repositories

4. Testing con Repositories (15%)

Excelente (100%)

```
# Test con repository mock
class FakeTaskRepository:
    def __init__(self):
        self.tasks = {}
        self._id = 1

    def create(self, task: Task) -> Task:
        task.id = self._id
        self.tasks[self._id] = task
        self._id += 1
        return task

def test_create_task():
    fake_repo = FakeTaskRepository()
    service = TaskService(task_repo=fake_repo)

    task = service.create_task(TaskCreate(title="Test"))

    assert task.id == 1
    assert task.title == "Test"
```

-  Tests con fake repositories
-  Service testado sin base de datos
-  Cobertura de casos de éxito y error

Bueno (75%)

-  Tests funcionales
-  Pocos casos cubiertos

Suficiente (50%)

-  Al menos 1 test con mock
-  Tests dependen de BD real

Insuficiente (< 50%)

-  Sin tests

- ❌ Tests no usan repositories



Escala de Calificación

Nivel	Rango	Descripción
🌟 Excelente	90-100%	Dominio completo del patrón
✅ Bueno	75-89%	Implementación sólida con mejoras menores
⚠️ Suficiente	60-74%	Cumple requisitos mínimos
❌ Insuficiente	< 60%	No cumple criterios básicos



Proyecto: Task Manager API

Requisitos Mínimos (60%)

- ☐ BaseRepository con métodos CRUD
- ☐ TaskRepository heredando de base
- ☐ UserRepository heredando de base
- ☐ TaskService usando repositorios
- ☐ Endpoints funcionando

Requisitos Completos (80%)

- ☐ Todos los anteriores
- ☐ Type hints completos
- ☐ Manejo de errores con excepciones custom
- ☐ Al menos 3 tests unitarios

Requisitos Avanzados (100%)

- ☐ Todos los anteriores
- ☐ Unit of Work implementado
- ☐ Tests con fake repositories
- ☐ Documentación en código



Checklist de Entrega

Estructura del Proyecto

```
task-manager/  
├─ main.py  
├─ config.py  
├─ database.py  
├─ models/  
│   ├── __init__.py  
│   ├── user.py  
│   └─ task.py  
├─ schemas/  
│   ├── __init__.py  
│   ├── user.py  
│   └─ task.py
```

```

├── repositories/           ← NUEVO
│   ├── __init__.py
│   ├── base.py           ← BaseRepository
│   ├── user_repository.py
│   └── task_repository.py
├── services/
│   ├── __init__.py
│   └── task_service.py
├── routers/
│   ├── __init__.py
│   ├── users.py
│   └── tasks.py
└── tests/                 ← NUEVO
    ├── __init__.py
    └── test_task_service.py

```

Verificación

- ☐ Código ejecuta sin errores
- ☐ Tests pasan: `pytest tests/`
- ☐ Endpoints funcionan en `/docs`
- ☐ No hay imports de `sqlalchemy` en `services/`



Errores Comunes a Evitar

1. Repository con lógica de negocio

```

# ❌ MAL - validación en repository
class TaskRepository:
    def create(self, task: Task) -> Task:
        if task.due_date < date.today():
            raise ValueError("Invalid date") # ← NO aquí
        ...

```

2. Service con queries SQLAlchemy

```

# ❌ MAL - SQLAlchemy en service
class TaskService:
    def get_pending(self):
        return self.db.query(Task).filter(Task.done == False).all()

```

3. Repository sin tipado

```

# ❌ MAL - sin type hints
def get_by_id(self, id): # ← falta tipo de retorno
    return self.db.get(self.model, id)

```



Recursos de Apoyo

- [Repository Pattern - Fowler](#)
- [Python Generics](#)
- [pytest Documentation](#)
